

Tugas 3.: Tugas Praktikum Mandiri

Oryza Ayunda Putri - 0110224030

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: nasi.tektekmangudin@gmail.com

1. Tugas Praktikum Mandiri

1.1 Memanggil Library Python

Bagian ini digunakan untuk memanggil berbagai **library Python** yang diperlukan dalam proses *Machine Learning*:

- **pandas** → membaca dan mengelola data dalam bentuk tabel (CSV).
- **numpy** → melakukan operasi matematis (misalnya array dan vektor).
- **matplotlib** dan **seaborn** → membuat grafik visualisasi data dan hasil evaluasi.
- **train_test_split** → membagi dataset menjadi data latih dan data uji.
- **LabelEncoder** → mengubah data kategori menjadi numerik.
- **StandardScaler** → melakukan normalisasi data agar setiap fitur memiliki skala yang sama.
- **SVC** → algoritma *Support Vector Classification* dari *Scikit-Learn*.
- **accuracy_score**, **classification_report**, **confusion_matrix** → mengukur performa model.
- **PCA** → mereduksi dimensi data agar bisa divisualisasikan 2D.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.decomposition import PCA
```

✓ 0.0s

Gambar 1 Memanggil Library Python

1.2 Load Dataset

Tahapan ini bertujuan untuk **membaca dataset** fitness.csv ke dalam *DataFrame pandas*.

- **df.head()** menampilkan 5 baris pertama agar kita tahu struktur data.
- **df.info()** menampilkan tipe data dan jumlah nilai non-null setiap kolom.
- **df.describe()** memberikan ringkasan statistik seperti rata-rata (*mean*), standar deviasi, nilai minimum dan maksimum.

Hasil tahap ini membantu memastikan data lengkap dan siap untuk dianalisis.

```
df = pd.read_csv("../data/fitness.csv")

print("=== 5 Data Teratas ===")
display(df.head())

print("\n=== Informasi Dataset ===")
print(df.info())

print("\n=== Statistik Deskriptif ===")
display(df.describe())
```

✓ 0.0s

Gambar 2 Load Dataset

=== 5 Data Teratas ===

	record_id	timestamp	user_id	age	gender	activity_type	duration_min	calories_burned	heart_rate_avg	steps	sleep_hours	weight_kg
0	1	2025-07-22 09:22:54	U026	67	Male	Cycling	7	322	175	3591	7.0	84.3
1	2	2025-05-28 13:53:54	U039	16	Female	Strength Training	83	265	113	22312	5.7	79.6
2	3	2025-08-07 01:36:54	U046	62	Female	Cycling	118	508	98	5418	6.6	88.3
3	4	2025-07-17 01:56:54	U048	56	Female	Swimming	75	270	150	19809	7.9	69.0
4	5	2025-08-11 03:22:54	U042	42	Other	Yoga	76	308	141	12562	5.0	61.6

Gambar 3 Dataset Teratas

```
=== Informasi Dataset ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   record_id             200 non-null   int64
1   timestamp              200 non-null   object
2   user_id                200 non-null   object
3   age                    200 non-null   int64
4   gender                 200 non-null   object
5   activity_type          200 non-null   object
6   duration_min           200 non-null   int64
7   calories_burned        200 non-null   int64
8   heart_rate_avg         200 non-null   int64
9   steps                  200 non-null   int64
10  sleep_hours            200 non-null   float64
11  weight_kg              200 non-null   float64
dtypes: float64(2), int64(6), object(4)
memory usage: 18.9+ KB
None
```

Gambar 4 Informasi Dataset

=== Statistik Deskriptif ===

	record_id	age	duration_min	calories_burned	heart_rate_avg	steps	sleep_hours	weight_kg
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	44.380000	64.025000	295.765000	120.935000	14020.82000	6.313000	70.152000
std	57.879185	15.417982	34.693987	120.563834	34.100366	6428.05981	1.911518	14.129562
min	1.000000	16.000000	5.000000	50.000000	60.000000	370.00000	3.000000	35.300000
25%	50.750000	33.000000	33.000000	220.000000	89.750000	9585.00000	4.775000	61.450000
50%	100.500000	43.000000	68.000000	311.500000	121.000000	14857.00000	6.500000	70.600000
75%	150.250000	58.000000	95.000000	378.000000	150.250000	19007.75000	7.900000	80.100000
max	200.000000	69.000000	119.000000	585.000000	179.000000	24984.00000	9.500000	105.200000

Gambar 5 Statistik Deskriptif

1.3 Membuat Kolom Target Klasifikasi

Karena dataset *Fitness* awalnya tidak memiliki kolom kategori (label), maka dibuat kolom *Calorie_Level* berdasarkan jumlah kalori yang dibakar.

- `pd.cut()` membagi data `calories_burned` menjadi 3 kategori:
 - Low untuk ≤ 200 kalori
 - Medium untuk 200–400 kalori
 - High untuk > 400 kalori
- Hasil ini menjadi **label target** untuk model klasifikasi SVM.

```
df['Calorie_Level'] = pd.cut(
    df['calories_burned'],
    bins=[0, 200, 400, float('inf')],
    labels=['Low', 'Medium', 'High']
)

print("\n=== Jumlah Data per Kategori ===")
print(df['Calorie_Level'].value_counts())

display(df[['calories_burned', 'Calorie_Level']].head(10))
```

✓ 0.0s

Gambar 6 Membuat Kolom Target Klasifikasi

```
=== Jumlah Data per Kategori ===
Calorie_Level
Medium    121
Low       41
High      38
Name: count, dtype: int64
```

	calories_burned	Calorie_Level
0	322	Medium
1	265	Medium
2	508	High
3	270	Medium
4	308	Medium
5	471	High
6	238	Medium
7	133	Low
8	500	High
9	210	Medium

Gambar 7 Output

1.4 Pisahkan Fitur (X) dan Label (y)

- Langkah ini memisahkan:
 - Fitur (X):** semua kolom selain Calorie_Level
 - Label (y):** kolom Calorie_Level
- Kemudian jika ada kolom bertipe teks (object), diubah ke angka menggunakan **LabelEncoder()**, karena model SVM hanya bisa membaca data numerik.

```
target_col = 'Calorie_Level'

X = df.drop(columns=[target_col])
y = df[target_col]

for col in X.select_dtypes(include=['object']).columns:
    X[col] = LabelEncoder().fit_transform(X[col])
```

✓ 0.0s

Gambar 8 Pisahkan Fitur (X) dan Label (y)

1.5 Split Dataset dan Normalisasi

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

✓ 0.0s

Gambar 9 Split Dataset dan Normalisasi

Dataset dibagi menjadi:

- 80% data latih (training set)
- 20% data uji (testing set)

Kemudian dilakukan normalisasi (scaling) menggunakan **StandardScaler()** agar setiap fitur memiliki distribusi data yang seragam (rata-rata 0 dan standar deviasi 1). Ini penting karena SVM sensitif terhadap skala fitur.

1.6 Bangun dan Latih Model SVM

```
model = SVC(kernel='rbf', gamma='scale', C=1.0)

model.fit(X_train, y_train)
```

✓ 0.0s

▼ SVC ⓘ ?

► Parameters

Gambar 10 Bangun dan Latih Model SVM

- Membuat model SVM dengan parameter:
 - kernel='rbf': kernel *Radial Basis Function* yang cocok untuk data non-linear.
 - C=1.0: parameter regularisasi untuk mengontrol margin.
 - gamma='scale': mengatur sensitivitas pengaruh data latih terhadap hyperplane.
- model.fit() melatih model agar mengenali pola antara fitur dan label.

1.7 Prediksi dan Evaluasi Model

```

y_pred = model.predict(X_test)

print("Akurasi:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

✓ 0.0s

Akurasi: 0.825

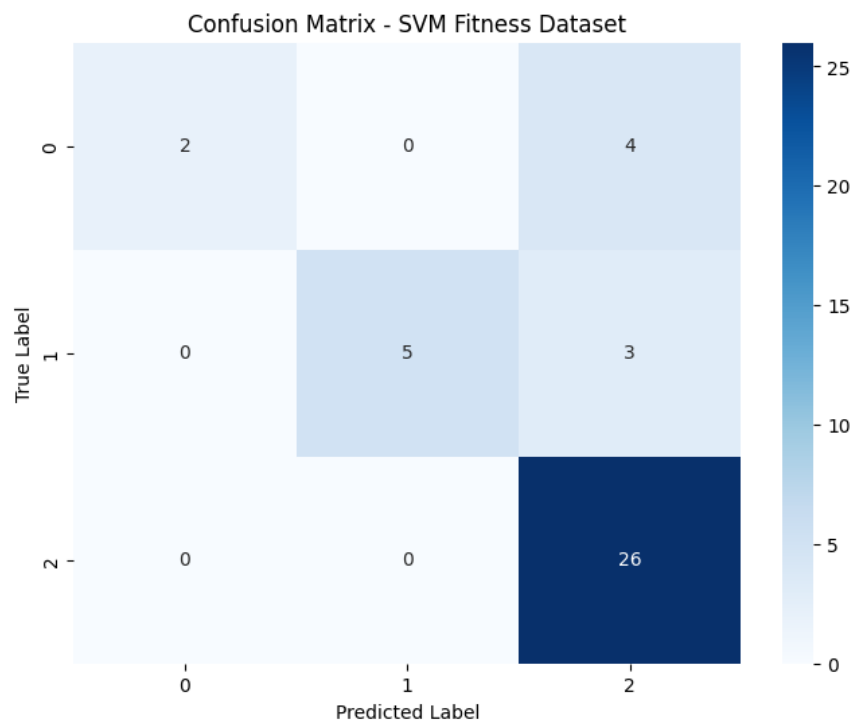
	precision	recall	f1-score	support
High	1.00	0.33	0.50	6
Low	1.00	0.62	0.77	8
Medium	0.79	1.00	0.88	26
accuracy			0.82	40
macro avg	0.93	0.65	0.72	40
weighted avg	0.86	0.82	0.80	40

Gambar 11 Prediksi dan Evaluasi Model

- **Akurasi (Accuracy) = 0.825**
 Artinya model berhasil mengklasifikasikan data dengan tingkat ketepatan **82,5%**, yang tergolong **baik** untuk dataset sederhana seperti ini.
 Dengan kata lain, dari 40 data uji, sekitar 33 data berhasil diklasifikasikan dengan benar.
- **Precision, Recall, dan F1-score:**
 - **Precision** menunjukkan seberapa banyak prediksi kelas yang benar dibandingkan total prediksi kelas tersebut.
 Nilai tinggi (mendekati 1.00) menunjukkan sedikit kesalahan prediksi.
 - **Recall** menunjukkan seberapa banyak data aktual dari suatu kelas yang berhasil dikenali oleh model.
 - **F1-score** adalah kombinasi dari *precision* dan *recall*, memberikan ukuran keseimbangan antara keduanya.
- **Kelas “Medium” memiliki performa terbaik**, dengan nilai:
 - Precision = 0.79
 - Recall = 1.00

- F1-score = 0.88
Ini berarti model sangat baik dalam mengenali kategori *Medium* (tidak ada yang salah klasifikasi ke kelas lain).
- **Kelas “High” memiliki recall terendah (0.33)**
Artinya model kesulitan mengenali data *High Calorie* dan cenderung mengklasifikasikannya ke kategori lain. Hal ini bisa disebabkan jumlah data *High* yang lebih sedikit (hanya 6 data dari total 40).
- **Macro Average dan Weighted Average:**
 - *Macro avg* = rata-rata tanpa mempertimbangkan jumlah data per kelas.
 - *Weighted avg* = rata-rata dengan mempertimbangkan proporsi data tiap kelas.
Nilai *weighted avg f1-score* = 0.80 menandakan kinerja model stabil di semua kelas.

1.8 Confusion Matrix



Gambar 12 Confusion Matrix

- Nilai diagonal (2, 5, 26) menunjukkan jumlah data yang terklasifikasi dengan benar untuk tiap kelas.
- Semakin besar nilai diagonal, semakin baik performa model dalam mengenali kelas tersebut.
- Model SVM dengan kernel RBF ini bekerja sangat baik untuk kategori High, cukup baik untuk Medium, namun kurang optimal untuk Low.

1.9 Visualisasi PCA 2D

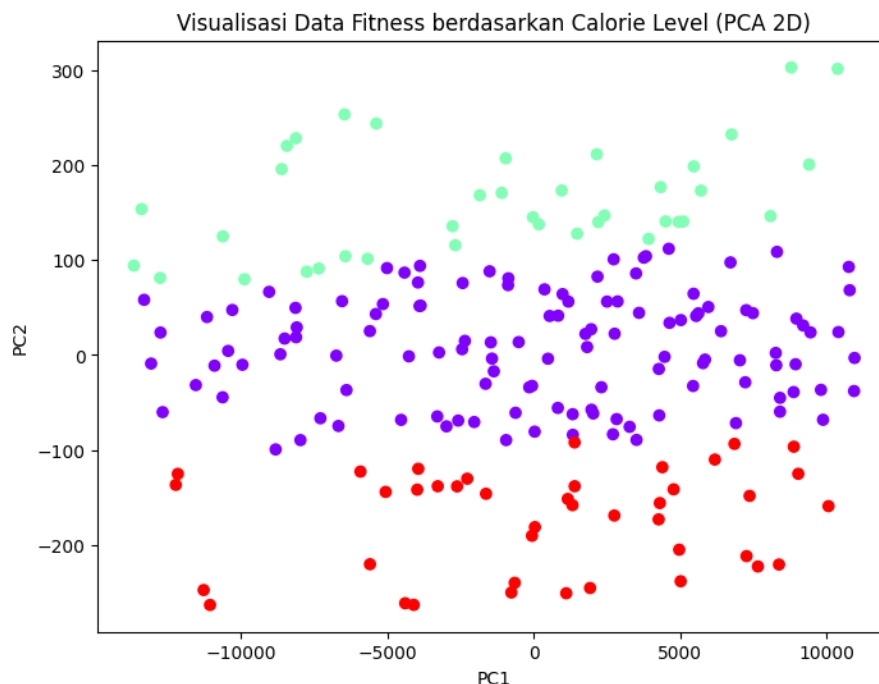
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.figure(figsize=(8,6))
plt.scatter(X_pca[:,0], X_pca[:,1], c=pd.factorize(y)[0], cmap='rainbow')
plt.title('Visualisasi Data Fitness berdasarkan Calorie Level (PCA 2D)')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Gambar 13 Visualisasi PCA 2D

Hasil PCA 2D menunjukkan bahwa:

- Data fitness berdasarkan fitur-fitur seperti usia, durasi aktivitas, detak jantung, dan langkah harian memiliki pola distribusi yang cukup jelas antara kategori *Low*, *Medium*, dan *High* Calorie Level.
- Hal ini memperkuat bukti bahwa model SVM mampu melakukan klasifikasi dengan akurasi yang baik (82,5%) sebagaimana diperoleh pada tahap evaluasi.



Gambar 14 Visualisasi PCA

1.10 Visualisasi 3D

```
from mpl_toolkits.mplot3d import Axes3D
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

x1 = df['age']
x2 = df['duration_min']
x3 = df['heart_rate_avg']

fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')

sc = ax.scatter(x1, x2, x3, c=y_encoded, cmap='rainbow', s=60, alpha=0.8)

ax.set_xlabel('Age')
ax.set_ylabel('Duration (min)')
ax.set_zlabel('Heart Rate Avg')
ax.set_title('3D Visualisasi Data Fitness berdasarkan Calorie Level (SVM Classification)')

colors = [plt.cm.rainbow(i / len(encoder.classes_)) for i in range(len(encoder.classes_))]
for color, label in zip(colors, encoder.classes_):
    ax.scatter([], [], [], color=color, label=label)

ax.legend(title="Calorie Level", loc='upper left')

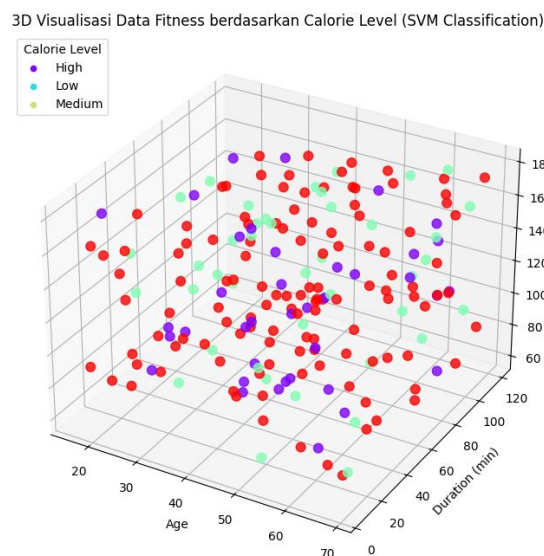
plt.show()
```

Gambar 15 Visualisasi 3D

Bagian ini menampilkan hasil klasifikasi dalam bentuk grafik 3 dimensi:

- Sumbu X, Y, Z mewakili tiga fitur utama: *Age*, *Duration*, dan *Heart Rate*.
- Warna titik menunjukkan kategori kalori: *Low*, *Medium*, *High*.
- Dengan *Axes3D*, pengguna dapat memutar tampilan grafik untuk melihat pemisahan antar kelas dari berbagai sudut.

Visualisasi ini membantu memahami distribusi data dan kemampuan model dalam memisahkan tiap kelas di ruang tiga dimensi.



Gambar 16 Hasil Visualisasi 3D

Kesimpulan:

Secara keseluruhan, percobaan ini membuktikan bahwa algoritma Support Vector Machine (SVM) mampu bekerja efektif untuk kasus klasifikasi multikelas pada data kebugaran dengan tingkat akurasi yang tinggi (82,5%). Dengan preprocessing yang baik dan pemilihan kernel yang tepat, model SVM dapat mempelajari hubungan antara variabel kebugaran seperti usia, durasi latihan, detak jantung, dan kalori terbakar untuk mengelompokkan tingkat aktivitas seseorang secara akurat.

Link:

- <https://www.kaggle.com/datasets/prince7489/fitness-tracker-dataset>
- https://github.com/orzaynd/machine_learning/tree/master/praktikum/praktikum6
- https://github.com/orzaynd/machine_learning/tree/master/tugas